

Diseño orientado a objetos en Java

SOLID

Cinco principios para escribir código fácil de cambiar

S Responsabilidad única

Una clase debe tener una sola razón para cambiar.

Antes

```
class Factura {  
    double calcularTotal() {...}  
    void imprimir() {...}  
    void guardarEnBD() {...}  
}
```

Después

```
class Factura {  
    double calcularTotal() {...}  
}  
class ImpresoraFactura { ... }  
class RepositorioFactura { ... }
```

Abierto / Cerrado

Abierto para extender, cerrado para modificar.

Antes

```
double descuento(String tipo) {  
    if (tipo.equals("VIP"))  
        return 0.2;  
    if (tipo.equals("NAVIDAD"))  
        return 0.1;  
    return 0;  
}
```

Después

```
interface Descuento {  
    double valor();  
}  
class DescuentoVip  
    implements Descuento {...}  
class DescuentoNavidad  
    implements Descuento {...}
```

L Sustitución de Liskov

Una subclase debe poder usarse donde se espera su clase padre.

Antes

```
class Ave {  
    void volar() {...}  
}  
class Pinguino extends Ave {  
    void volar() {  
        throw new Exception();  
    }  
}
```

Después

```
class Ave { ... }  
interface Volador {  
    void volar();  
}  
class Gorrión extends Ave  
    implements Volador {...}  
class Pinguino extends Ave {...}
```

Segregación de interfaces

Mejor varias interfaces pequeñas que una grande.

Antes

```
interface Trabajador {  
    void trabajar();  
    void comer();  
}  
class Robot implements Trabajador {  
    // ¿y comer()?  
}
```

Después

```
interface Trabajable {  
    void trabajar();  
}  
interface Alimentable {  
    void comer();  
}  
class Robot implements Trabajable  
class Humano implements  
    Trabajable, Alimentable
```

D Inversión de dependencias

Depender de abstracciones, no de clases concretas.

Antes

```
class Aviso {  
    EnviadorEmail envio =  
        new EnviadorEmail();  
}  
// Atado al email
```

Después

```
interface Enviador {  
    void enviar(String msg);  
}  
class Aviso {  
    private Enviador envio;  
    Aviso(Enviador e) {  
        envio = e;  
    }  
}
```

Resumen

S **Responsabilidad única**

Una razón para cambiar

O **Abierto / Cerrado**

Se extiende sin modificar

L **Liskov**

El hijo sustituye al padre

I **Segregación de interfaces**

Contratos pequeños

D **Inversión de dependencias**

Depender de abstracciones